SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology

in

Computer Science & Engineering

By

Aman Singh B (VTU24376)

10211CS224 - Full Stack Application Development (Java)

Slot: E3-B19

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled **"Smart Campus Event Management System"** submitted by **Aman Singh B (VTU24376)**, Reg. No. **23UECS0046**, has been carried out in the Winter Semester of Academic Year 2025–2026 under the course **10211CS224 - Full Stack Application Development (Java)**.

Signature of Faculty
Dr. X. Arogya Presskila
Assistant Professor
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
April, 2026

Signature of course coordinator
Dr. M. Sumithra
Assistant Professor (Senior Grade)
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission. I understand that any violation of the above will be cause for disciplinary action by the Institute.

Aman Singh B
VTU24376
Date:06 April, 2026

ABSTRACT

The Smart Campus Event Management System is a full-stack web application developed to simplify the management of college events such as workshops, seminars, hackathons, and campus activities. In many institutions, event details are shared through notice boards, WhatsApp groups, classroom announcements, or manual forms, which often leads to missed information, low participation, and poor coordination between students and administrators.

This project addresses these challenges by providing a centralized digital platform where all campus events are listed in one place. Students can browse upcoming events, filter them by department or event type, register online by entering their name and email, and view all their registered events by simply entering their email address. No student login is required, making the process quick and accessible.

The system also provides a secure admin module protected by Spring Security. Administrators can add new events, edit existing event details, delete old events, search events with filters, and view real-time registration statistics showing how many students registered for each event. The backend is developed using Java 17, Spring Boot, Spring MVC, Spring Data JPA, and H2 Database. The frontend is implemented using Thymeleaf, HTML5, CSS3, and JavaScript with a clean and responsive design.

As an innovation module, a Campus Event Assistant Chatbot has been added. The chatbot allows students to ask questions about upcoming events, venues, departments, registration steps, and admin login guidance. The chatbot uses keyword matching to provide instant responses.

The project follows MVC architecture with proper separation of concerns. Key features include CRUD operations, form validation, global exception handling using @ControllerAdvice, REST API support, and role-based access control. The project supports SDG 4 - Quality Education by improving access to academic and skill-development events for all students.

Keywords: Spring Boot, Thymeleaf, Event Management, H2 Database, Spring Security, REST API, Chatbot, MVC Architecture, CRUD Operations, SDG 4, Campus Events, Workshop Management

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DTO	Data Transfer Object
H2	H2 Database (In-Memory Database)
HTML	HyperText Markup Language
JDK	Java Development Kit
JPA	Java Persistence API
JSON	JavaScript Object Notation
MVC	Model View Controller
ORM	Object Relational Mapping
RAM	Random Access Memory
REST	Representational State Transfer
SDG	Sustainable Development Goal
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

A Smart Campus Event Management System is a web-based platform that helps students and administrators manage campus events in one centralized place. Campus events include workshops, seminars, hackathons, cultural programs, technical sessions, placement training activities, and guest lectures.

In many colleges today, event information is still shared through traditional methods such as physical notice boards, WhatsApp groups, classroom announcements, or Google Forms. While these methods are simple, they create several problems. Students may miss important updates because notices are posted only in specific locations or messages get lost in crowded WhatsApp groups. Administrators struggle to track registrations manually using spreadsheets or emails, which is time-consuming and error-prone. When event details change, there is no efficient way to notify all interested students.

This project solves these problems by providing a single digital platform where all events are published, students can register online, and administrators can manage everything from a secure dashboard. The system eliminates paper forms, reduces manual work, and ensures that no student misses an important event due to lack of information.

The project is built using modern full-stack Java technologies including Spring Boot for the backend, Thymeleaf for dynamic web pages, and H2 in-memory database for data storage. Spring Security protects admin pages, and Spring Data JPA handles all database operations without writing SQL queries.

1.2 Problem Statement

Many colleges and universities still rely on manual and semi-digital methods for event communication and registration. The specific problems observed are:

1. **Lack of Centralized Information:** Event information is scattered across notice boards, WhatsApp groups, classroom announcements, and emails. Students have to check multiple sources to find events, and often miss important opportunities.
2. **Manual Registration Process:** Students register by sending their names to class representatives or filling Google Forms. There is no confirmation whether their registration was received or not. Duplicate registrations and missing entries are common.
3. **Difficult Participant Tracking:** Administrators track participants using Excel sheets, which becomes messy when dealing with multiple events. Analyzing participation data for each event requires manual effort.
4. **No Real-Time Updates:** When an event date, venue, or time changes, there is no efficient way to notify all registered students. Some students come at the wrong time while others miss the event entirely.
5. **No Registration Statistics:** Administrators cannot easily see which events are popular or how many students registered for each event without manually counting entries.
6. **Wasted Administrative Effort:** Coordinators spend hours on manual paperwork, follow-ups, and data entry instead of focusing on creating quality events for students.

Therefore, a reliable, user-friendly, and centralized Smart Campus Event Management System is needed to bridge this gap. The system should provide students with a simple way to discover and register for events and give administrators complete control to manage events and track participation in real time.

1.3 Aim of the Project

The aim of this project is to design, develop, and deploy a full-stack web application that allows students to discover, register, and track campus events while allowing administrators to manage event details securely and view registration statistics in real time.

1.4 Objectives

The main objectives of this project are:

- To design and develop a web-based Smart Campus Event Management System using Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, and H2 Database.
- To implement two user roles with clear separation of responsibilities – Students (browse and register) and Administrators (manage events and view statistics).
- To allow students to browse all upcoming events, filter them by department and event type, and register online by entering their name and email address.
- To allow students to view all their registered events by simply entering their email address, without needing to create a separate login account.
- To provide administrators with a secure login page protected by Spring Security.
- To enable administrators to perform complete CRUD operations – add new events, edit existing events, delete old events, and search events with filters.
- To display real-time registration statistics on the admin dashboard showing how many students registered for each event.
- To implement global exception handling using @ControllerAdvice for a smooth user experience.
- To implement a Campus Event Assistant Chatbot as an innovation module to help students with event-related queries.

- To demonstrate full-stack Java web development with MVC architecture, validation, security, and database integration.

1.5 Scope of the Project

The Smart Campus Event Management System is designed for colleges and universities that conduct multiple academic and non-academic events throughout the year. The system can be used by:

- **Students:** To discover upcoming events, filter by department or type, register online, and track their registered events.
- **Administrators/Event Coordinators:** To create events, update event details, delete events, search events, and view registration statistics.

The current version of the project uses H2 in-memory database, which makes it perfect for academic demonstration, testing, and development. No separate database installation is required. The system runs on any modern web browser and requires no client-side installation.

The project can be extended in the future to support features like email notifications, QR code attendance, persistent databases like MySQL, student login with profiles, and mobile application support.

1.6 Technologies Used

The project uses the following technologies:

- **Backend:** Java 17, Spring Boot, Spring MVC, Spring Data JPA, Spring Security
- **Frontend:** Thymeleaf, HTML5, CSS3, JavaScript
- **Database:** H2 In-Memory Database
- **Build Tool:** Maven
- **IDE:** Eclipse IDE

- **Hardware Requirements:** 4GB RAM, i3 Processor, 20GB Storage
- **Software Requirements:** Windows 10/11/Linux/macOS, Java JDK 17+, Modern Web Browser

1.7 SDG Alignment

This project supports **SDG 4 – Quality Education**. By making educational events like workshops, seminars, and hackathons easily discoverable and accessible to all students, the system helps remove barriers to skill development. Students can attend events that supplement their classroom learning, gain practical skills, and improve their employability. The system is inclusive and does not require any special access, ensuring equal opportunities for all students.

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

2.1 Existing Event Management Methods

Before developing this system, a survey was conducted to understand how colleges currently manage events. The following methods are commonly used:

2.1.1 Physical Notice Boards

Paper notices are posted on bulletin boards around the campus. While this method is simple and low-cost, it has major drawbacks. Students may not see the notice if they do not pass by the board. Notices can be torn down or covered by other notices. There is no way to track how many students saw the notice or registered for the event.

2.1.2 WhatsApp Groups

Event information is shared in class or department WhatsApp groups. This is fast and reaches many students quickly. However, messages get lost in the flood of daily chats. Not all students may read the message. There is no organized way to collect registrations or track participants.

2.1.3 Google Forms

Google Forms are used to collect registrations. This method provides organized data in a spreadsheet. However, administrators still need to manually analyze the data. There is no real-time dashboard to see registration counts. Students cannot see their own registration status easily. Multiple forms for different events create confusion.

2.1.4 Excel Sheets

Administrators maintain Excel sheets to track participants. This is useful for storing lists but is completely manual. Updating sheets, removing duplicate entries, and sending updates to students takes significant time and effort.

2.1.5 Third-Party Event Platforms

Some colleges use external platforms like Eventbrite or Townscript. These platforms are powerful but often charge fees. They are not specifically designed for campus events and may have features that are not needed.

2.2 Comparison of Existing Systems

Various methods are currently used for event management in colleges, each with its own advantages and limitations (Table 2.1).

Method	**Advantages**	**Limitations**	**Cost**
Notice Board	Simple, no setup	Students may miss notices, no tracking	Free
WhatsApp Groups	Fast communication	Messages get lost, no organized tracking	Free
Google Forms	Easy data collection	No real-time dashboard, manual analysis	Free
Excel Sheets	Good for storage	Manual, error-prone, time-consuming	Free
Eventbrite	Professional platform	Costly, not campus-specific	Paid
Our System	Centralized, real-time stats, secure, free	H2 database (temporary)	Free

Table 2.1: Comparison of Existing Event Management Methods

It is observed from Table 2.1 that existing methods have significant limitations. Notice boards are easily missed, WhatsApp messages get lost, Google Forms require manual analysis, Excel sheets are error-prone, and paid platforms like Eventbrite are costly and not campus-specific.

2.3 Need for Proposed System

From the survey, it is clear that existing methods solve only part of the problem and are often not integrated (Table 2.1). The proposed Smart Campus Event Management System combines event listing, online registration, admin management, real-time statistics, database storage, and chatbot assistance in one platform. This reduces manual work, improves student participation, and provides a seamless experience for both students and administrators.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the application is implemented using Thymeleaf templates integrated with Spring Boot. Thymeleaf is a server-side Java template engine that allows dynamic data from backend controllers to be rendered directly into HTML pages. The frontend also uses HTML5 for structure, CSS3 for styling and responsive design, and JavaScript for interactive elements including form validation and the chatbot widget.

3.1.2 Structure of the Frontend

The main frontend files organized in the `src/main/resources/templates` directory are:

- `index.html` – Home page displaying welcome banner, Smart Campus tagline, and two main buttons for students and admins.
- `events.html` – Event listing page where students can view all upcoming events. Includes filter options for department (CSE, ECE, ME, etc.) and event type (Workshop, Seminar, Hackathon, Cultural).
- `event-details.html` – Detailed view of a single event with complete information and a registration form.

- `register.html` – Registration form page where students enter name and email to register for an event.
- `my-registrations.html` – Page where students can enter their email address to view all events they have registered for.
- `admin/login.html` – Admin login page protected by Spring Security.
- `admin/dashboard.html` – Admin dashboard showing event list, add event form, edit/delete buttons, and registration statistics.
- `admin/event-form.html` – Form for adding or editing event details.
- `error.html` – Custom error page for handling exceptions gracefully.

3.1.3 Execution Procedure

When a user opens the application in a web browser, Spring Boot serves the Thymeleaf templates. The backend controllers add data to the model, and Thymeleaf renders the HTML with the actual data. For example, when a student visits `/events`, the StudentController fetches all events from the database, adds them to the model, and returns `events.html`. Thymeleaf then loops through the events and displays each one as a card.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Java 17 and Spring Boot framework. Maven is used as the build tool for dependency management. The project dependencies are defined in `pom.xml` and include Spring Boot Starter Web, Spring Boot Starter Data JPA, Spring Boot Starter Thymeleaf, Spring Boot Starter Security, H2 Database, and Spring Boot DevTools.

3.2.2 Structure of the Backend

The backend follows a layered architecture with the following packages:

- **Controller Layer:**
  - `HomeController.java` – Handles home page requests at `/`.
  - `StudentController.java` – Handles event browsing, filtering, registration, and registration lookup at `/events`, `/register`, `/my-registratio`
  - `AdminController.java` – Handles admin login, dashboard, add event, edit event, delete event, search events, and statistics at `/admin/**`.
  - `EventRestController.java` – Provides REST API endpoints at `/api/even`
  - `ChatbotController.java` – Provides REST API for chatbot at `/api/chatb`
  - `GlobalExceptionHandler.java` – Global exception handling using @ControllerAdvice.
- **Service Layer:**
  - `EventService.java` – Contains business logic for event CRUD operations.
  - `RegistrationService.java` – Contains business logic for registrations.
  - `ChatbotService.java` – Contains logic for generating chatbot responses.
- **Repository Layer:**
  - `EventRepository.java` – JPA repository for Event entity.
  - `EventRegistrationRepository.java` – JPA repository for EventRegistration entity.
- **Model Layer:**
  - `Event.java` – Entity class representing EVENTS table.
  - `EventRegistration.java` – Entity class representing EVENT_REGISTRATIO table.
  - `EventType.java` – Enum for event types.
- **Configuration Layer:**

- `SecurityConfig.java` – Spring Security configuration for admin authentication.
- `DataInitializer.java` – Loads sample events when application starts.

3.2.3 Default Admin Credentials

The system uses Spring Security for admin authentication. The default login credentials are:

- **Username:** admin
- **Password:** admin123

3.2.4 Execution Procedure

The application entry point is `SmartCampusEventManagementApplication.ja` which contains the `main` method with `SpringApplication.run()`. When executed:

1. Spring Boot starts an embedded Tomcat server on port 8080.
2. The H2 in-memory database is created automatically.
3. Spring Data JPA creates tables based on entity classes.
4. DataInitializer inserts sample events into the database.
5. The application is ready to accept HTTP requests.

3.3 Database Implementation

3.3.1 Database Configuration

The project uses H2 in-memory database. The configuration is defined in `src/main/re`

```
# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:campusevents
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA Configuration
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# H2 Console Configuration
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Server Configuration
server.port=8080
```

Listing 3.1: application.properties Configuration

3.3.2 Schema Structure

The database contains two main tables:

EVENTS Table

Stores all event details. The fields are:

- `id` – Primary key, auto-generated
- `title` – Event title (e.g., ”AI Workshop 2026”)
- `description` – Detailed description of the event
- `date` – Date of the event
- `department` – Department conducting the event (CSE, ECE, ME, etc.)
- `eventType` – Type of event (Workshop, Seminar, Hackathon, Cultural)

- `venue` – Location where event is conducted
- `capacity` – Maximum number of participants

EVENT_REGISTRATIONS Table

Stores student registration records. The fields are:

- `id` – Primary key, auto-generated
- `studentName` – Name of the registered student
- `email` – Email address of the student
- `eventId` – Foreign key referencing EVENTS table
- `registrationTime` – Timestamp when registration was submitted

The database consists of two main tables as shown in (Table 3.1).

Table Name	Purpose
EVENTS	Stores all campus event details including title, description, date, department, type, venue, capacity
EVENT_REGISTRATIONS	Stores student registration records including student name, email, event reference

Table 3.1: Database Tables

The structure and purpose of each table are described in Table 3.1. The EVENTS table stores all event details, while the EVENT_REGISTRATIONS table stores student registration records linked to events via the eventId foreign key.

3.3.3 H2 Database Console

The H2 database console is available at `http://localhost:8080/h2-console`. To log in:

- JDBC URL: `jdbc:h2:mem:campusevents`
- Username: `sa`
- Password: *leave blank*

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a layered architecture with six distinct layers. Each layer has a specific responsibility and communicates only with adjacent layers (Figure 4.1).

4.1.1 Client Layer

The Client Layer consists of the web browser where students and administrators access the application. Students can view events, register, and check registrations. Administrators can log in and manage all events from the admin dashboard.

4.1.2 Controller Layer

The Controller Layer handles all incoming HTTP requests using Spring MVC. HomeController manages the home page. StudentController handles event browsing, filtering, registration, and registration lookup. AdminController handles login, add event, edit event, delete event, search events, and registration statistics.

4.1.3 Service Layer

The Service Layer contains all business logic of the application. EventService manages event CRUD operations and search functionality. RegistrationService manages student registrations, email-based registration lookup, duplicate registration prevention, and registration statistics.

4.1.4 Repository Layer

The Repository Layer uses Spring Data JPA to interact with the database without writing SQL queries. EventRepository and EventRegistrationRepository provide methods for data operations.

4.1.5 Database Layer

The Database Layer uses H2 in-memory database with EVENTS and EVENT_REGISTRA tables.

4.1.6 Security Layer

The Security Layer uses Spring Security to protect all admin pages under the /admin path.

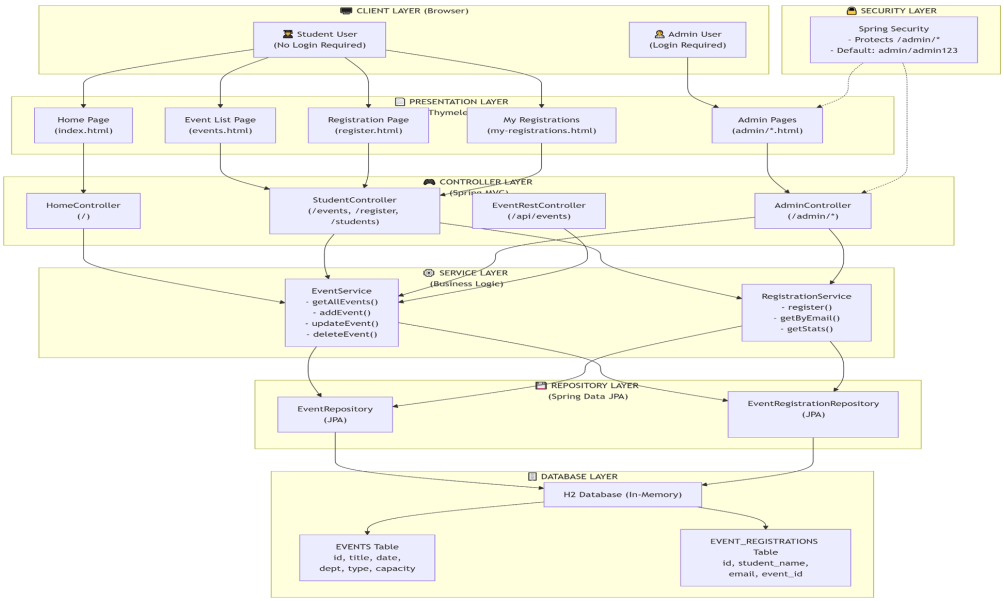

Figure 4.1: System Architecture Diagram

The layered architecture of the system is illustrated in Figure 4.1. Each layer communicates only with the layers directly above and below it, ensuring loose coupling and maintainability.

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how data moves through the system (Figure 4.2). It shows the flow of information between students, administrators, and the system components.

4.2.1 Level 0 DFD (Context Diagram)

At the highest level, the system has two external entities: Student and Admin. The student interacts with the system to view events, register, and check registrations. The admin interacts with the system to manage events and view statistics.

4.2.2 Level 1 DFD

The Level 1 DFD breaks down the main processes:

- Process 1: Event Management (Admin adds, edits, deletes events)
- Process 2: Event Browsing (Student views and filters events)
- Process 3: Registration Management (Student registers, system stores data)
- Process 4: Statistics Generation (System calculates registration counts)
- Data Store 1: EVENTS Table
- Data Store 2: EVENT_REGISTRATIONS Table

4.2.3 Data Flow Description

The basic data flow in the system is as follows:

1. Administrator logs in using Spring Security authentication.
2. Admin creates a new event – data saved to H2 database via EventRepository.
3. Student opens the events page – StudentController fetches events from database.
4. Events are displayed to the student using Thymeleaf.

5. Student registers for an event – registration saved to EVENT_REGISTRATIONS table.
6. Student can view registered events by entering email.
7. Admin can view registration statistics on the dashboard.

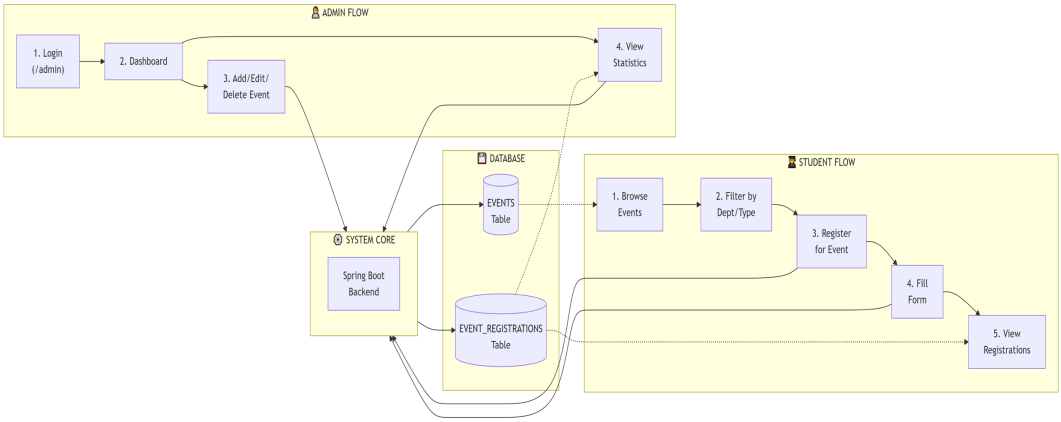

Figure 4.2: Data Flow Diagram of Smart Campus Event Management System

The complete data flow of the system is shown in Figure 4.2, which maps how data travels from user input through controllers, services, repositories, and finally to the database.

4.3 Module Descriptions

4.3.1 Module 1: User Authentication and Security Module

This module handles admin login using Spring Security. All admin pages under the /admin path are protected. Default credentials are username ”admin” and password ”admin123”. Student pages are public and do not require any authentication. Spring Security uses BCrypt password encoding for secure storage. The module also manages session handling and logout functionality.

4.3.2 Module 2: Student Event Browsing and Filtering Module

This module allows students to view all upcoming events in a clean card layout. Students can filter events by department (CSE, ECE, ME, Civil, etc.) and by event

type (Workshop, Seminar, Hackathon, Cultural). The filtering is implemented using custom query methods in EventRepository. Each event card displays the event title, description, date, department, type, venue, and available capacity. The module also supports sorting events by date.

4.3.3 Module 3: Student Registration Module

This module handles student registration for events. When a student clicks the Register button on an event, a form opens asking for name and email. Both fields are validated to ensure they are not empty and email format is correct. The system checks if the student is already registered for the same event using the email address. If already registered, an error message is shown. If not registered, the registration record is saved to the EVENT_REGISTRATIONS table with a timestamp. A success message is displayed and the student is given options to go back or view all registrations.

4.3.4 Module 4: Student Registration Lookup Module

This module allows students to view all events they have registered for without needing a password. On the My Registrations page, students enter their email address. The system fetches all registration records matching that email from the EVENT_REGISTRATIONS table, joins with the EVENTS table to get event details, and displays the list of registered events. Each entry shows the event title, date, department, type, and registration time. If no registrations are found, a friendly message says ”No registrations found for this email.”

4.3.5 Module 5: Admin Event Management Module

This module provides complete CRUD functionality for administrators:

- **Create (Add Event):** Admin fills a form with title, description, date, department, event type, venue, and capacity. All fields are validated. On submission, the event is saved to the EVENTS table.
- **Read (View Events):** Admin can see all events in a list with all details.

- **Update (Edit Event):** Admin clicks the Edit button on any event, a pre-filled form opens, updates are made, and changes are saved to the database.
- **Delete (Delete Event):** Admin clicks the Delete button, a confirmation dialog appears, and upon confirmation the event is removed from the EVENTS table. All associated registrations are also removed to maintain referential integrity.
- **Search:** Admin can search events by department and event type using filter options.

4.3.6 Module 6: Statistics and Reporting Module

This module displays real-time registration statistics on the admin dashboard. For each event, the system shows the total number of registrations received by counting records in the EVENT_REGISTRATIONS table where eventId matches. The dashboard also shows:

- Total number of events in the system
- Total number of registrations across all events
- Most popular event (highest registrations)
- Event-wise registration counts in a table format

The statistics are calculated in real-time every time the dashboard is loaded or refreshed.

4.3.7 Module 7: Innovation Module – Campus Event Assistant Chatbot

The innovation module is a Campus Event Assistant Chatbot available on all student-facing pages. It appears as a floating chat button at the bottom right corner of the screen. When clicked, a chat window opens. Students can type questions in natural language. The chatbot analyzes keywords in the question and provides appropriate responses. Key features include:

- Answers questions about upcoming events and how to view them
- Explains the registration process step by step

- Provides admin login credentials when asked
- Gives information about event venues and departments
- Responds to greetings like ”Hello” and ”Hi”
- Handles unknown queries with a helpful default message

The chatbot uses a keyword-matching algorithm in the backend ChatbotService. It does not require any external API keys and works entirely within the application.

4.4 Advantages of the Project

- Centralized platform for all campus events – no more notice boards or WhatsApp forwards
- Online registration eliminates paper forms and manual tracking
- Real-time registration statistics help admin track participation easily
- Spring Security protects admin pages from unauthorized access
- No installation required – works in any web browser with H2 database running automatically
- Students can view registered events by simply entering email – no login required
- Chatbot provides instant answers to common student questions
- REST API support for future mobile app integration
- Global exception handling ensures smooth user experience
- Lightweight and fast due to H2 in-memory database
- Easy to demonstrate in academic settings
- Complete CRUD operations with validation

4.5 Limitations of the Project

- H2 in-memory database loses all data when the application stops running
- No student login system – students are identified only by email
- No email notifications for registration confirmation or event reminders
- No export functionality for downloading registration lists as PDF or Excel
- No QR code check-in system to verify student attendance
- Single admin account only – no multiple admin roles
- Chatbot uses basic keyword matching, not advanced AI
- No payment gateway integration for paid events
- No mobile application – works only on web browsers
- No event calendar view – only list view available

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Home Page

The home page is the first screen users see at `http://localhost:8080`. It displays a welcome banner showing ”Smart Campus” with a tagline explaining that the platform manages campus events, workshops, and seminars in one place (Figure 5.1). Two buttons are available – ”Explore Upcoming Events” for students and ”Open Admin Console” for administrators. The ”UPCOMING LINEUP” section displays featured events loaded from the database.

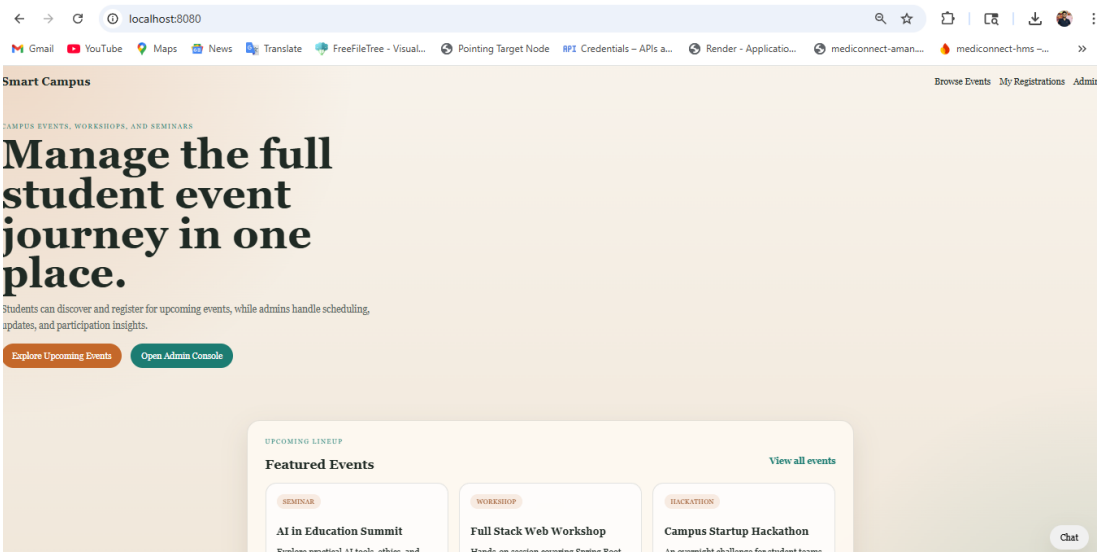

Figure 5.1: Home Page of Smart Campus Event Management System

The home page layout is shown in Figure 5.1, which serves as the entry point for all users.

5.2 Event Listing Page

The event listing page at `/events` displays all upcoming events (Figure 5.2). Filter options allow students to search by department and event type. Each event card shows title, description, date, department, type, and capacity. Every event has a ”Register” button.

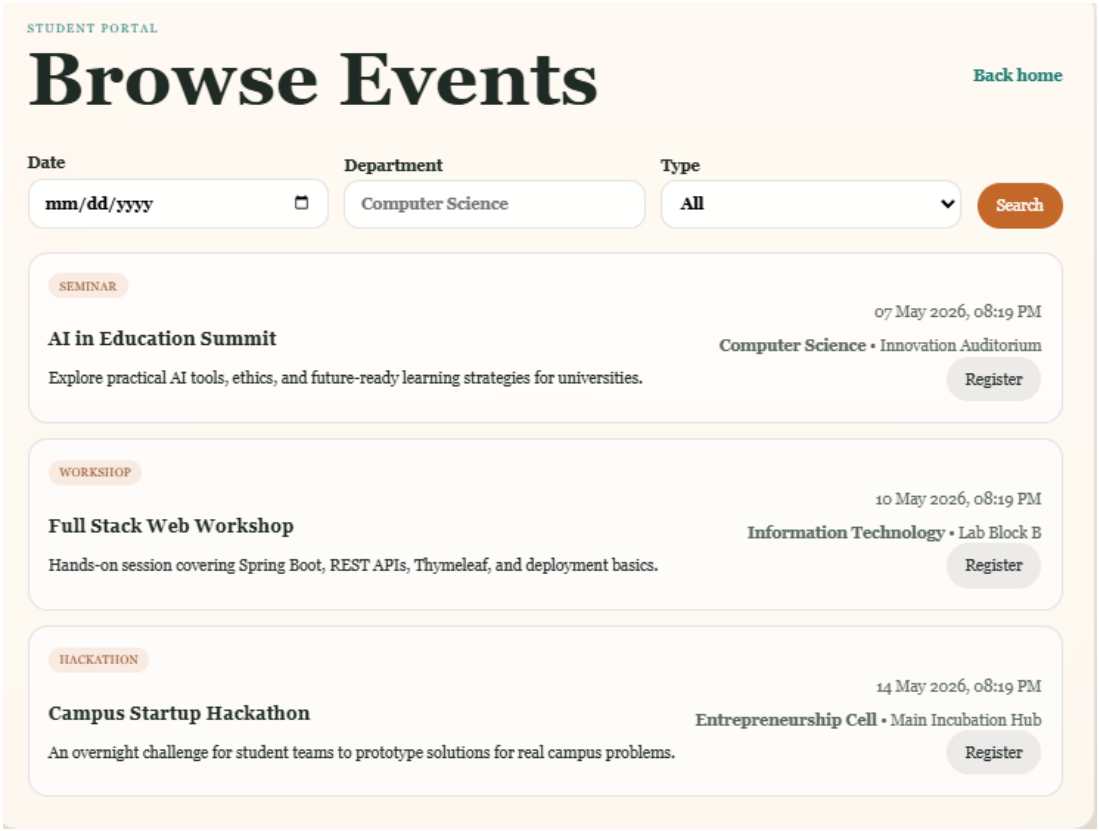

Figure 5.2: Event Listing Page with Filter Options

Students can browse and filter all available events from the page shown in Figure 5.2.

5.3 Student Registration Page

When a student clicks ”Register”, the registration page opens with event details and a form for name and email (Figure 5.3). Both fields are required. The system prevents duplicate registration.

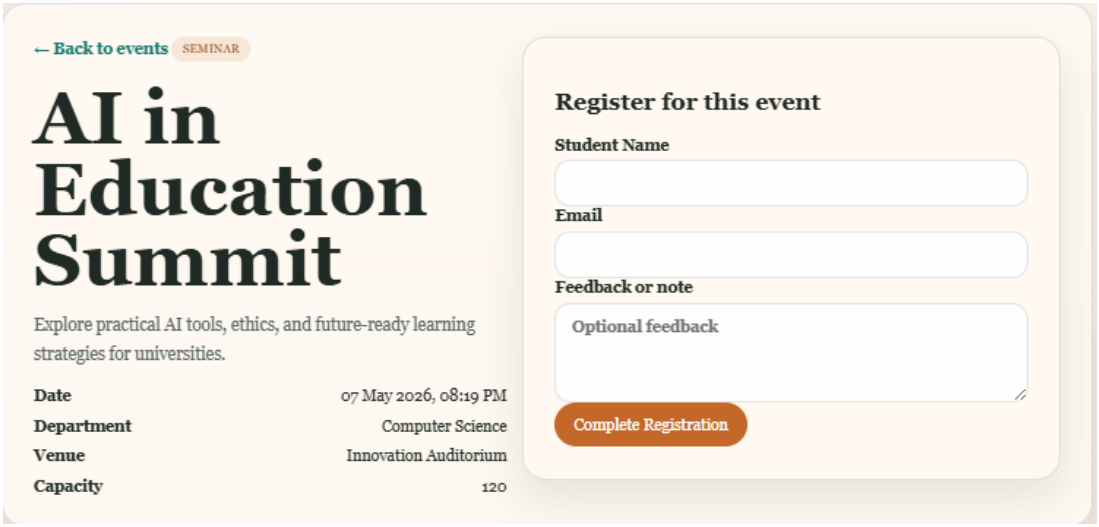

Figure 5.3: Student Event Registration Page

The registration process is illustrated in Figure 5.3.

5.4 Registered Events Page (My Registrations)

The My Registrations page at `/my-registrations` allows students to enter their email and view all events they have registered for (Figure 5.4).

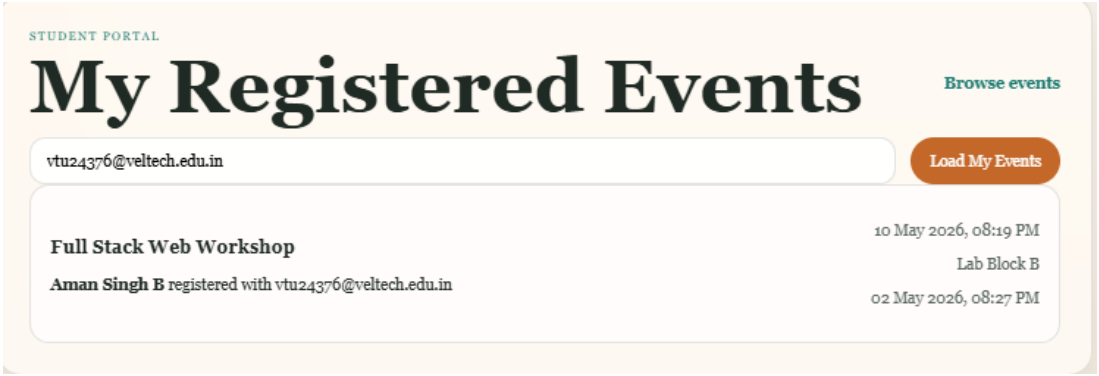

Figure 5.4: Student Registered Events Page

Students can track their registered events using the page shown in Figure 5.4.

5.5 Admin Login Page

The admin login page at `/admin` is protected by Spring Security (Figure 5.5). Default credentials are username ”admin” and password ”admin123”.

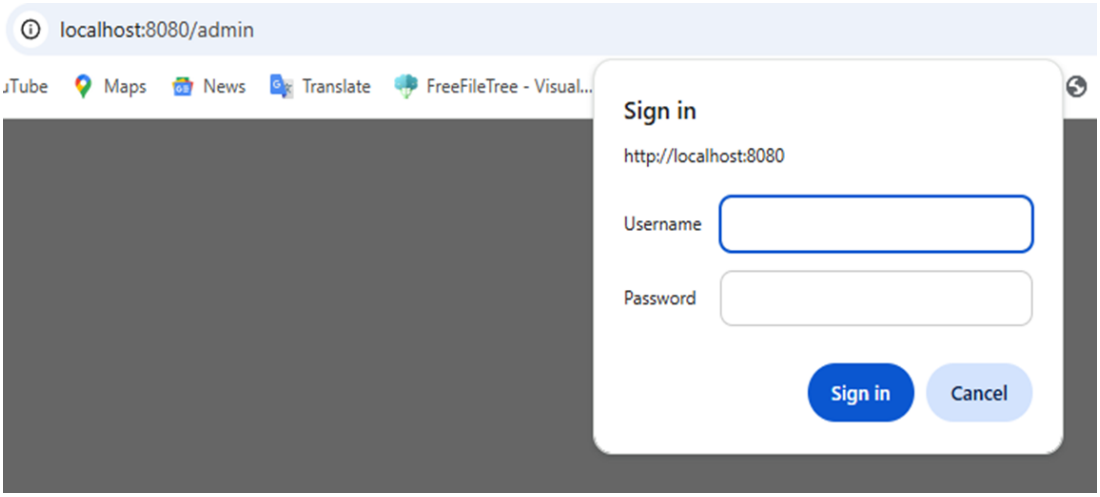

Figure 5.5: Admin Login Page

The secure admin login interface is depicted in Figure 5.5.

5.6 Admin Dashboard

The admin dashboard shows add event form, event list with edit/delete buttons, and registration statistics (Figure 5.6). Admin can add, edit, delete, and search events.

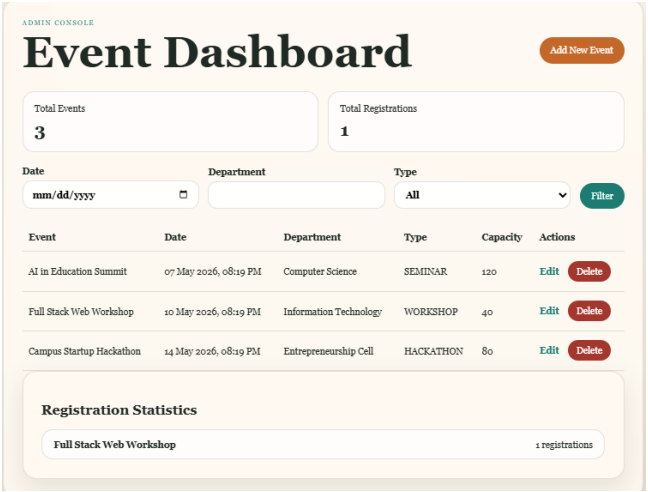

Figure 5.6: Admin Dashboard with Event Management and Statistics

The admin dashboard (Figure 5.6) provides complete control over event management.

5.7 H2 Database Console

The H2 console at `/h2-console` allows viewing EVENTS and EVENT_REGISTRATIO tables (Figure 5.7 and Figure 5.8).

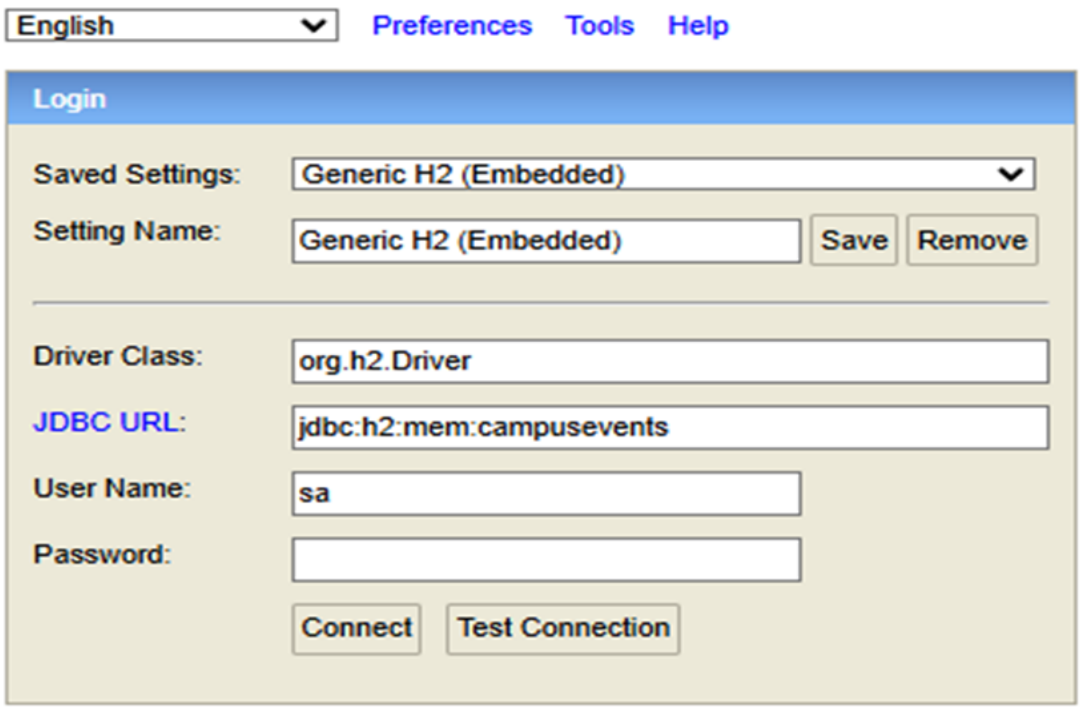

Figure 5.7: H2 Database Console Login Page

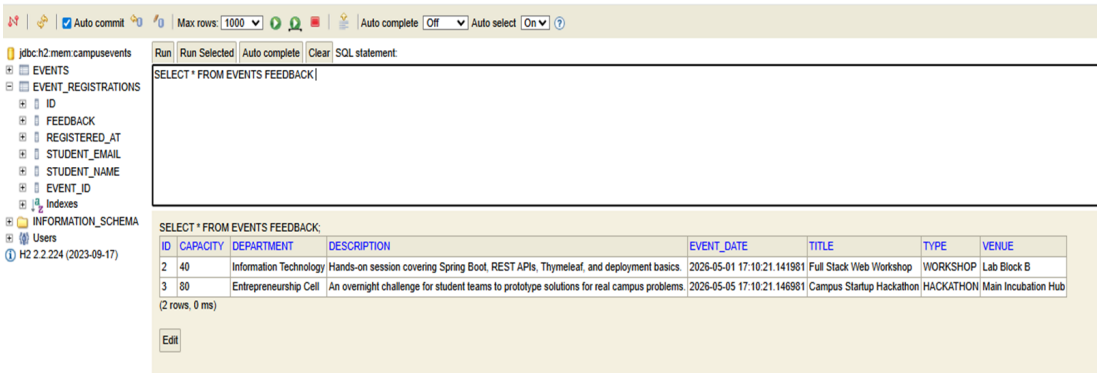

Figure 5.8: H2 Database Console Showing Tables

The H2 database login page is shown in Figure 5.7, and the table viewer is shown in Figure 5.8.

5.8 Chatbot Innovation Module

The Campus Event Assistant Chatbot appears as a floating button on student pages (Figure 5.9). Students can ask questions about events, registration, venues, and admin login.

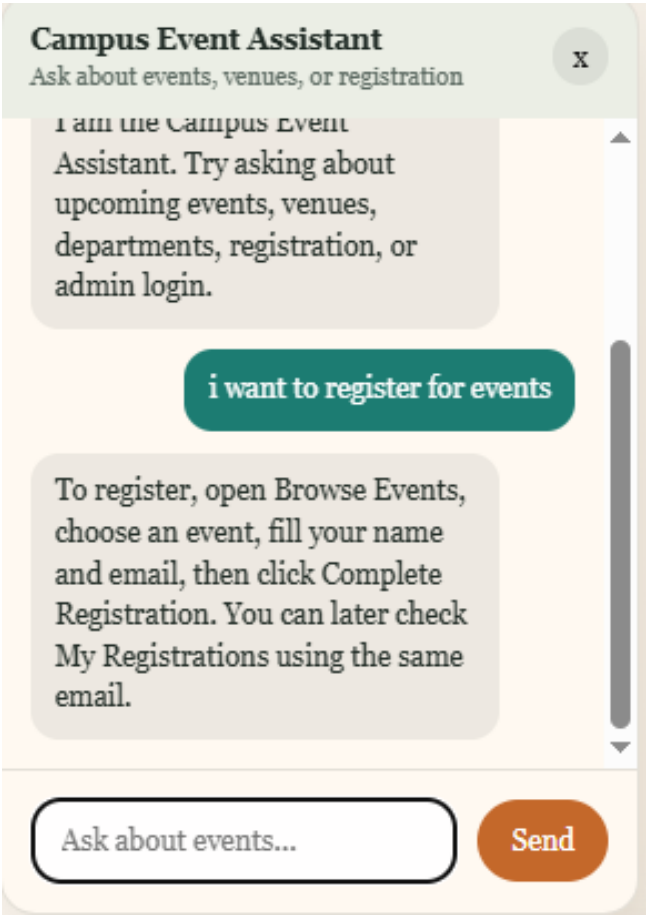

Figure 5.9: Campus Event Assistant Chatbot

The chatbot interface (Figure 5.9) provides instant answers to student queries.

5.9 Result Summary Table

The summary of all implemented features and their results is presented in Table 5.1.

Feature	**Result**
Student Event Browsing	Students can view all upcoming events with search and filters
Student Registration	Students can register online with name, email, and validation
Duplicate Registration Prevention	System prevents same email from registering twice for same event
Registration Lookup by Email	Students can view registered events by entering email
Admin Authentication	Spring Security protects all admin pages
Admin Add Event	New events saved to database and appear on student pages
Admin Edit Event	Event details can be updated using pre-filled form
Admin Delete Event	Events removed from database with confirmation
Admin Search Events	Admins can search events by department and event type
Registration Statistics	Admin dashboard shows event-wise registration counts
H2 Database Integration	EVENTS and EVENT_REGISTRATIONS tables store all data
Global Exception Handling	User-friendly error pages instead of technical errors
REST API	GET /api/events returns JSON data successfully
Chatbot Innovation	Students can ask event-related questions and get responses

Table 5.1: Implemented Features and Results Summary

All features listed in Table 5.1 have been successfully implemented and tested.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System has been successfully designed, developed, and tested as a full-stack web application for managing campus events. The project successfully demonstrates the integration of various Spring Boot modules including Spring MVC for request handling, Spring Data JPA for database operations, Spring Security for admin authentication, and Thymeleaf for server-side page rendering.

The system achieves all its stated objectives. Students can browse upcoming events, filter them by department or event type, register online, and view all their registered events by entering their email. Administrators can log in securely, add new events, edit existing events, delete old events, search events, and view real-time registration statistics.

The innovation module – Campus Event Assistant Chatbot – adds interactive value to the project. Students can ask questions about events, registration, venues, and admin login, and receive instant responses.

The project supports SDG 4 – Quality Education – by making educational events more accessible to all students. Overall, this project successfully demonstrates full-stack Java web development skills including MVC architecture, database integration, CRUD operations, form validation, security, REST API development, exception handling, and frontend-backend integration.

6.2 Future Enhancements

The following enhancements can be implemented in the future:

- **Persistent Database:** Replace H2 in-memory database with MySQL or PostgreSQL for permanent data storage.
- **Student Login and Profiles:** Implement student registration and login using Spring Security with personal profiles.
- **Email Notifications:** Integrate JavaMailSender to send confirmation emails after registration and reminder emails before events.
- **QR Code Attendance:** Generate unique QR codes for each registration to mark attendance at the event venue.
- **Export Functionality:** Add options to export registration lists as PDF, Excel, or CSV.
- **Event Calendar View:** Replace list view with an interactive calendar view.
- **Payment Integration:** Integrate payment gateways like Razorpay for paid events.
- **Multiple Admin Roles:** Implement role-based access for different levels of administrators.
- **Advanced Chatbot:** Upgrade chatbot to use AI APIs like Google Dialogflow for natural language understanding.
- **Mobile Application:** Develop a mobile app using React Native or Flutter that consumes the REST APIs.
- **Event Feedback System:** Allow students to rate and review events they attended.
- **Certificate Generation:** Automatically generate participation certificates in PDF format.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The mapping of the project to various Complex Engineering Problem (CEP) attributes is presented in Table 7.1 and Table 7.1.

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of knowledge	Requires in depth engineering knowledge	WK3,WK4,WK5,WK6	Requires knowledge in Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies (Thymeleaf, HTML/CSS) for developing a scalable event management system.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4,WK5,WK6	Balances usability and security by ensuring easy student access while maintaining secure admin authentication and data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3,WK4,WK5	Involves system design, database schema creation, validation handling, and efficient event filtering and registration mechanisms.
WP4	Familiarity	Partially new problems	WK4,WK8	Integrating real-time event updates, user interaction, and admin analytics introduces challenges beyond traditional manual systems.

Level	Attribute	Characteristics	WKs	Justification
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event filtering, registration tracking, and statistics generation not fully covered by standard templates.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5,WK6,WK7	Addresses requirements of students, faculty coordinators, and administrators ensuring smooth event organization and participation.
WP7	Interdependence	System level integration	WK3,WK5,WK6	Integrates frontend UI, backend services, database, validation, and security modules into a unified system.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Table 7.1 covers WP1 to WP4, while Table 7.1 covers WP5 to WP7 of the CEP mapping.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

The mapping of the project to Complex Engineering Activities (CEA) is shown in Table 7.2.

EA No.	**Attribute**	**Characteristics**	**Justification based on the Project**
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

All five CEA attributes (EA1 to EA5) are addressed by the project as described in Table 7.2.

7.3 SDG Mapping (CEP Section)

The Sustainable Development Goals (SDG) mapping for the project is presented in Table 7.3.

SDG No.	**SDG Title**	**Relevance to the Project**
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management.

Table 7.3: SDG Mapping for the Project

As shown in Table 7.3, the project contributes to five SDGs, with primary focus on SDG 4 (Quality Education).

REFERENCES

1. Spring Boot Official Documentation. https://spring.io/projects/spring-boot
2. Spring Security Reference Documentation. https://spring.io/projects/spring-security
3. Spring Data JPA Documentation. https://spring.io/projects/spring-dat
4. Thymeleaf Official Documentation. https://www.thymeleaf.org/documenta html
5. H2 Database Engine Documentation. https://www.h2database.com/html/main.html
6. Baeldung. Spring Boot Tutorials. https://www.baeldung.com/spring-boot
7. MDN Web Docs. HTML, CSS, JavaScript Documentation. https://developer.mozilla.org/
8. United Nations. Sustainable Development Goal 4: Quality Education. https://sdgs.un.org/goals/goal4